

Assignment 5

Nischal Pant

Date submitted: 2023-11-26

In []: 9. In this exercise, we will predict the number of applications received using the other variables in the College data set.
(a) Split the data set into a training set and a test set.

```
In [1]: import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.decomposition import PCA
from sklearn.linear_model import RidgeCV, LassoCV
from sklearn.cross_decomposition import PLSRegression
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import RFE
from sklearn.pipeline import make_pipeline
from sklearn.feature_selection import RFE

college_data = pd.read_csv('college.csv')
```

```
In [2]: print(college_data.head())
X = college_data.drop('Apps', axis=1)
y = college_data['Apps']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=
```

		Unnamed: 0	Private	Apps	Accept	Enroll	Top10perc	\
0	Abilene Christian University	Yes	1660	1232	721	23		
1	Adelphi University	Yes	2186	1924	512	16		
2	Adrian College	Yes	1428	1097	336	22		
3	Agnes Scott College	Yes	417	349	137	60		
4	Alaska Pacific University	Yes	193	146	55	16		

	Top25perc	F.Undergrad	P.Undergrad	Outstate	Room.Board	Books	Personal	\
0	52	2885	537	7440	3300	450	2200	
1	29	2683	1227	12280	6450	750	1500	
2	50	1036	99	11250	3750	400	1165	
3	89	510	63	12960	5450	450	875	
4	44	249	869	7560	4120	800	1500	

	PhD	Terminal	S.F.Ratio	perc.alumni	Expend	Grad.Rate
0	70	78	18.1	12	7041	60
1	29	30	12.2	16	10527	56
2	53	66	12.9	30	8735	54
3	92	97	7.7	37	19016	59
4	76	72	11.9	2	10922	15

In []: (b) Fit a linear model using least squares on the training set, and report the test error obtained.

```
In [3]: categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
X_encoded = pd.get_dummies(X, columns=categorical_cols)

X_train, X_test, y_train, y_test = train_test_split(X_encoded, y, test_size=0.2, random_state=42)
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
linear_test_predictions = linear_model.predict(X_test)
linear_test_error = mean_squared_error(y_test, linear_test_predictions)
print("Linear Regression Test Error:", linear_test_error)
```

Linear Regression Test Error: 1491544.9737312628

```
In [ ]: (c) Fit a ridge regression model on the training set, with  $\lambda$  chosen by cross-validation. Report the test error obtained.
```

```
In [4]: alphas = np.logspace(-6, 6, 13)
ridge_model = RidgeCV(alphas=alphas, store_cv_values=True)

ridge_model.fit(X_train, y_train)
optimal_alpha = ridge_model.alpha_
ridge_test_predictions = ridge_model.predict(X_test)

ridge_test_error = mean_squared_error(y_test, ridge_test_predictions)
print("Optimal Alpha ( $\lambda$ ):", optimal_alpha)
print("Ridge Regression Test Error:", ridge_test_error)
```

Optimal Alpha (λ): 10.0

Ridge Regression Test Error: 1483881.314107998

```
In [ ]: (d) Fit a lasso model on the training set, with  $\lambda$  chosen by crossvalidation. Report the test error obtained.
```

```
In [5]: alphas = np.logspace(-6, 6, 13)
lasso_model = LassoCV(alphas=alphas, cv=5)
lasso_model.fit(X_train, y_train)
optimal_alpha_lasso = lasso_model.alpha_
print("Optimal Alpha ( $\lambda$ ) for Lasso:", optimal_alpha_lasso)

lasso_test_predictions = lasso_model.predict(X_test)

lasso_test_error = mean_squared_error(y_test, lasso_test_predictions)
print("Lasso Regression Test Error:", lasso_test_error)

non_zero_coefficients = np.sum(lasso_model.coef_ != 0)
print("Number of Non-Zero Coefficient Estimates:", non_zero_coefficients)
```

```

C:\Users\Nischal\anaconda3\Lib\site-packages\sklearn\linear_model\_coordinate_descen
t.py:631: ConvergenceWarning: Objective did not converge. You might want to increase
the number of iterations, check the scale of the features or consider increasing regu
larisation. Duality gap: 1.023e+07, tolerance: 7.922e+05
    model = cd_fast.enet_coordinate_descent(
C:\Users\Nischal\anaconda3\Lib\site-packages\sklearn\linear_model\_coordinate_descen
t.py:631: ConvergenceWarning: Objective did not converge. You might want to increase
the number of iterations, check the scale of the features or consider increasing regu
larisation. Duality gap: 1.081e+06, tolerance: 7.922e+05
    model = cd_fast.enet_coordinate_descent(
C:\Users\Nischal\anaconda3\Lib\site-packages\sklearn\linear_model\_coordinate_descen
t.py:631: ConvergenceWarning: Objective did not converge. You might want to increase
the number of iterations, check the scale of the features or consider increasing regu
larisation. Duality gap: 1.089e+07, tolerance: 5.729e+05
    model = cd_fast.enet_coordinate_descent(
C:\Users\Nischal\anaconda3\Lib\site-packages\sklearn\linear_model\_coordinate_descen
t.py:631: ConvergenceWarning: Objective did not converge. You might want to increase
the number of iterations, check the scale of the features or consider increasing regu
larisation. Duality gap: 1.150e+06, tolerance: 5.729e+05
    model = cd_fast.enet_coordinate_descent(
C:\Users\Nischal\anaconda3\Lib\site-packages\sklearn\linear_model\_coordinate_descen
t.py:631: ConvergenceWarning: Objective did not converge. You might want to increase
the number of iterations, check the scale of the features or consider increasing regu
larisation. Duality gap: 1.163e+07, tolerance: 8.552e+05
    model = cd_fast.enet_coordinate_descent(
C:\Users\Nischal\anaconda3\Lib\site-packages\sklearn\linear_model\_coordinate_descen
t.py:631: ConvergenceWarning: Objective did not converge. You might want to increase
the number of iterations, check the scale of the features or consider increasing regu
larisation. Duality gap: 1.215e+06, tolerance: 8.552e+05
    model = cd_fast.enet_coordinate_descent(
C:\Users\Nischal\anaconda3\Lib\site-packages\sklearn\linear_model\_coordinate_descen
t.py:631: ConvergenceWarning: Objective did not converge. You might want to increase
the number of iterations, check the scale of the features or consider increasing regu
larisation. Duality gap: 1.167e+07, tolerance: 8.521e+05
    model = cd_fast.enet_coordinate_descent(
C:\Users\Nischal\anaconda3\Lib\site-packages\sklearn\linear_model\_coordinate_descen
t.py:631: ConvergenceWarning: Objective did not converge. You might want to increase
the number of iterations, check the scale of the features or consider increasing regu
larisation. Duality gap: 1.235e+06, tolerance: 8.521e+05
    model = cd_fast.enet_coordinate_descent(
C:\Users\Nischal\anaconda3\Lib\site-packages\sklearn\linear_model\_coordinate_descen
t.py:631: ConvergenceWarning: Objective did not converge. You might want to increase
the number of iterations, check the scale of the features or consider increasing regu
larisation. Duality gap: 9.973e+06, tolerance: 7.372e+05
    model = cd_fast.enet_coordinate_descent(
Optimal Alpha ( $\lambda$ ) for Lasso: 10.0
Lasso Regression Test Error: 1477246.3675593827
Number of Non-Zero Coefficient Estimates: 21

```

```

C:\Users\Nischal\anaconda3\Lib\site-packages\sklearn\linear_model\_coordinate_descen
t.py:631: ConvergenceWarning: Objective did not converge. You might want to increase
the number of iterations, check the scale of the features or consider increasing regu
larisation. Duality gap: 1.106e+06, tolerance: 7.372e+05
    model = cd_fast.enet_coordinate_descent(

```

In []: (e) Fit a PCR model on the training set, with M chosen by crossvalidation. Report the of M selected by cross-validation.

```

In [16]: components = np.arange(1, X_train.shape[1] + 1)
         cv_scores = []

```

```
# PCR MODEL
for component in components:
    try:
        pcr_model = make_pipeline(PCA(n_components=component), LinearRegression())
        scores = -cross_val_score(pcr_model, X_train, y_train, cv=10, scoring='neg_mean_squared_error')
        cv_scores.append(scores.mean())
    except ValueError:
        cv_scores.append(np.nan)

optimal_components = components[np.nanargmin(cv_scores)]
final_pcr_model = make_pipeline(PCA(n_components=optimal_components), LinearRegression())
final_pcr_model.fit(X_train, y_train)

pcr_test_predictions = final_pcr_model.predict(X_test)
pcr_test_error = mean_squared_error(y_test, pcr_test_predictions)

# Print the results
print(f"Optimal Number of Components (M): {optimal_components}")
print(f"PCR Test Error: {pcr_test_error}")
```

```
C:\Users\Nischal\anaconda3\Lib\site-packages\sklearn\model_selection\_validation.py:378: FitFailedWarning:
1 fits failed out of a total of 10.
The score on these train-test partitions for these parameters will be set to nan.
If these failures are not expected, you can try to debug them by setting error_score
='raise'.
```

Below are more details about the failures:

```

1 fits failed with the following error:
Traceback (most recent call last):
  File "C:\Users\Nischal\anaconda3\Lib\site-packages\sklearn\model_selection\_validation.py", line 686, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "C:\Users\Nischal\anaconda3\Lib\site-packages\sklearn\pipeline.py", line 401, in fit
    Xt = self._fit(X, y, **fit_params_steps)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Nischal\anaconda3\Lib\site-packages\sklearn\pipeline.py", line 359, in _fit
    X, fitted_transformer = fit_transform_one_cached(
                             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Nischal\anaconda3\Lib\site-packages\joblib\memory.py", line 349, in __call__
    return self.func(*args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Nischal\anaconda3\Lib\site-packages\sklearn\pipeline.py", line 893, in _fit_transform_one
    res = transformer.fit_transform(X, y, **fit_params)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Nischal\anaconda3\Lib\site-packages\sklearn\utils\_set_output.py", line 140, in wrapped
    data_to_wrap = f(self, X, *args, **kwargs)
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Nischal\anaconda3\Lib\site-packages\sklearn\decomposition\_pca.py", line 462, in fit_transform
    U, S, Vt = self._fit(X)
               ^^^^^^^^^^^
  File "C:\Users\Nischal\anaconda3\Lib\site-packages\sklearn\decomposition\_pca.py", line 512, in _fit
    return self._fit_full(X, n_components)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Nischal\anaconda3\Lib\site-packages\sklearn\decomposition\_pca.py", line 526, in _fit_full
    raise ValueError(
ValueError: n_components=559 must be between 0 and min(n_samples, n_features)=558 with svd solver='full'

```

```
warnings.warn(some_fits_failed_message, FitFailedWarning)
```

Optimal Number of Components (M): 64

PCR Test Error: 1491544.9737245857

```
In [ ]: (f) Fit a PLS model on the training set, with M chosen by crossvalidation. Report the
of M selected by cross-validation.
```

```
In [ ]: pip install statsmodels
```

```
vif = pd.DataFrame()
vif["Variable"] = X_train.columns
```

```

vif["VIF"] = [variance_inflation_factor(X_train.values, i) for i in range(X_train.shape[1])]

scaler = StandardScaler()
X_train_std = scaler.fit_transform(X_train)
X_test_std = scaler.transform(X_test)

# 10-fold validation
components = np.arange(1, X_train_std.shape[1] + 1)
cv_scores = []

# PLS MODEL AND CROSS VALIDATION
for component in components:
    pls_model = PLSRegression(n_components=component)
    scores = -cross_val_score(pls_model, X_train_std, y_train, cv=10, scoring='neg_mean_squared_error')
    cv_scores.append(scores.mean())

optimal_components = components[np.argmin(cv_scores)]

final_pls_model = PLSRegression(n_components=optimal_components)
final_pls_model.fit(X_train_std, y_train)

pls_test_predictions = final_pls_model.predict(X_test_std)
pls_test_error = mean_squared_error(y_test, pls_test_predictions)

print(f"Optimal Number of Components (M): {optimal_components}")
print(f"PLS Test Error: {pls_test_error}")

```

- In []: 11. We will now **try** to predict per capita crime rate **in** the Boston data set.
- (a) Try out some of the regression methods explored **in** this chapter, such **as** best subset selection, the lasso, ridge regression, **and** PCR. Present **and** discuss results **for** the approaches that you consider.
- (b) Propose a model (**or** set of models) that seem to perform well on this data set, **and** justify your answer. Make sure that you are evaluating model performance using validation set error, crossvalidation, **or** some other using training error.
- (c) Does your chosen model involve all of the features **in** the data set? Why **or** why not?

```

In [10]: model = LinearRegression()
selector = RFE(model, n_features_to_select=5)
selector = selector.fit(X_train, y_train)
selected_features = X_train.columns[selector.support_]

best_subset_model = LinearRegression()
best_subset_model.fit(X_train[selected_features], y_train)
best_subset_test_predictions = best_subset_model.predict(X_test[selected_features])
best_subset_test_error = mean_squared_error(y_test, best_subset_test_predictions)

```

```

In [11]: scaler = StandardScaler()
X_train_std = scaler.fit_transform(X_train)
X_test_std = scaler.transform(X_test)

lasso_model = LassoCV(cv=10)
lasso_model.fit(X_train_std, y_train)

```

```
lasso_test_predictions = lasso_model.predict(X_test_std)
lasso_test_error = mean_squared_error(y_test, lasso_test_predictions)
```

```
In [12]: ridge_model = RidgeCV(cv=10)
ridge_model.fit(X_train_std, y_train)
ridge_test_predictions = ridge_model.predict(X_test_std)
ridge_test_error = mean_squared_error(y_test, ridge_test_predictions)
```

```
In [13]: n_components = 10
pca = PCA(n_components=n_components)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

pcr_model = LinearRegression()
pcr_model.fit(X_train_pca, y_train)
pcr_test_predictions = pcr_model.predict(X_test_pca)
pcr_test_error = mean_squared_error(y_test, pcr_test_predictions)
```

```
In [14]: print("Results for Best Subset Selection:")
print(f"Selected Features: {selected_features}")
print(f"Test Error: {best_subset_test_error}\n")

print("Results for Lasso Regression:")
print(f"Selected Features (non-zero coefficients): {X_train.columns[lasso_model.coef_
print(f"Test Error: {lasso_test_error}\n")

print("Results for Ridge Regression:")
print(f"Test Error: {ridge_test_error}\n")

print("Results for Principal Component Regression (PCR):")
print(f"Number of Components: {n_components}")
print(f"Test Error: {pcr_test_error}")
```

Results for Best Subset Selection:

```
Selected Features: Index(['Unnamed: 0_Boston University',
                        'Unnamed: 0_Pennsylvania State Univ. Main Campus',
                        'Unnamed: 0_Purdue University at West Lafayette',
                        'Unnamed: 0_Rutgers at New Brunswick',
                        'Unnamed: 0_University of California at Berkeley'],
                        dtype='object')
```

Test Error: 13600158.844902864

Results for Lasso Regression:

```
Selected Features (non-zero coefficients): Index(['Accept', 'Top10perc', 'Room.Board',
                                                'Expend',
                                                'Unnamed: 0_Bloomsburg Univ. of Pennsylvania',
                                                'Unnamed: 0_Boston University', 'Unnamed: 0_Brown University',
                                                'Unnamed: 0_Columbia University', 'Unnamed: 0_Dartmouth College',
                                                'Unnamed: 0_Georgetown University', 'Unnamed: 0_Hampton University',
                                                'Unnamed: 0_James Madison University',
                                                'Unnamed: 0_Michigan State University',
                                                'Unnamed: 0_New York University', 'Unnamed: 0_Northwestern University',
                                                'Unnamed: 0_Pennsylvania State Univ. Main Campus',
                                                'Unnamed: 0_Princeton University',
                                                'Unnamed: 0_Purdue University at West Lafayette',
                                                'Unnamed: 0_Rutgers at New Brunswick',
                                                'Unnamed: 0_SUNY College at Brockport',
                                                'Unnamed: 0_SUNY College at Cortland',
                                                'Unnamed: 0_SUNY College at New Paltz', 'Unnamed: 0_SUNY at Binghamton',
                                                'Unnamed: 0_SUNY at Stony Brook', 'Unnamed: 0_Talladega College',
                                                'Unnamed: 0_University of California at Berkeley',
                                                'Unnamed: 0_University of Massachusetts at Amherst',
                                                'Unnamed: 0_University of Nebraska at Lincoln',
                                                'Unnamed: 0_University of North Carolina at Chapel Hill',
                                                'Unnamed: 0_University of Pennsylvania',
                                                'Unnamed: 0_University of Virginia', 'Private_No'],
                                                dtype='object')
```

Test Error: 1604611.8557821636

Results for Ridge Regression:

Test Error: 1611179.2040591284

Results for Principal Component Regression (PCR):

Number of Components: 10

Test Error: 1595725.9021476062

(b) Model Performance Evaluation and Proposal Based on the provided results: Best Subset Selection: The test error is relatively high (13,600,158.84). This indicates overfitting or that the selected features are not the most relevant for prediction. Lasso Regression: Lasso has a lower test error (1,604,611.86) and selected a subset of features, including both numerical and categorical variables related to university attributes. Ridge Regression: Test error is 1,611,179.20. It tends to keep all features but penalizes large coefficients, preventing overfitting. Principal Component Regression (PCR): PCR with 10 components has a test error of 1,595,725.90. It reduces dimensionality but still captures important information. Proposal: Among these models, Lasso Regression and PCR seem to perform reasonably well.

(c) Feature Involvement: Best Subset Selection: Involves a subset of universities, but the high test error suggests not all selected features are relevant. Lasso Regression: Lasso tends to perform feature selection by driving some coefficients to zero. Ridge Regression: Involves all features but penalizes large coefficients. It does not perform feature selection; rather, it shrinks coefficients toward zero. Principal Component Regression (PCR): Involves a reduced set of principal components instead of the original features, capturing the most important information in the data.